

## Soccer Training App - Comprehensive Project Documentation

### 🎯 Project Overview

A mobile application designed to help soccer players improve their skills through AI-powered video analysis. Users record themselves performing various soccer drills, and the app uses computer vision to analyze their technique, count touches, track performance metrics, and provide personalized feedback.

#### Core Functionality

- **Video Recording:** 30-second standardized drill recordings with countdown timer
- **AI Analysis:** Computer vision processing to detect ball touches, analyze technique
- **Performance Tracking:** Historical progress tracking with personal bests and trends
- **Detailed Feedback:** Comprehensive analysis with scores, insights, and improvement recommendations
- **Multi-Drill Support:** Currently supports juggling with framework for expansion

### 🏗️ Architecture Overview

#### Frontend (React Native/Expo)

- **Platform:** React Native with Expo SDK
- **Navigation:** React Navigation v6 with stack navigator
- **State Management:** React hooks (useState, useEffect)
- **Styling:** StyleSheet with consistent color palette
- **Icons:** @expo/vector-icons (MaterialIcons)
- **Camera:** expo-image-picker for video recording
- **HTTP Client:** Native fetch API with custom service layer
- **Authentication:** JWT token storage in memory (optional user system)

#### Backend (FastAPI/Python)

- **Location:** DigitalOcean Droplet (147.182.224.87:8000)
- **Framework:** FastAPI with Uvicorn ASGI server
- **Database:** SQLite with SQLAlchemy ORM
- **Authentication:** JWT tokens with optional authentication
- **File Storage:** Local filesystem for video uploads
- **Background Processing:** Python threading for async video analysis
- **CORS:** Enabled for mobile app access

#### Computer Vision Pipeline

- **Technology:** Custom Python CV analysis (details abstracted)
- **Processing:** Asynchronous analysis with status polling
- **Metrics Tracked:**
  - Touch count detection
  - Confidence scores
  - Per-foot analysis
  - Touch intervals
  - Detection frame details

### 📁 Project Structure

#### Frontend Structure

soccer-training-app-clean/

```
|— App.js           # Main app entry, navigation setup
|— app.json        # Expo configuration
|— package.json    # Dependencies
|— src/
|   |— screens/
|       |— HomeScreen.js      # Landing page with main CTAs
|       |— DrillSelectionScreen.js # Browse available drills
|       |— VideoRecordingScreen.js # Camera interface with countdown
|       |— AnalysisProgressScreen.js # Real-time analysis status
```

```

| | | | AnalysisResultsScreen.js # Comprehensive results display
| | | | DrillProgressScreen.js # Historical progress tracking
| | | |
| | | | services/
| | | | | api.js # Base API client with auth
| | | | | drills.js # Drill-specific API methods
| | | | |
| | | | constants/
| | | | | config.js # API endpoints, app config
| | | | |
| | | | assets/ # Images, fonts (if any)

```

#### Backend Structure

/root/soccerapp/backend/

```

| main.py # FastAPI app initialization
| models.py # SQLAlchemy models
| database.py # Database connection setup
| auth.py # JWT authentication
| drill_api.py # Drill analysis endpoints
| drill_progress_endpoint.py # Progress tracking endpoints
| simple_save_integration.py # Result saving logic
| cors_config.py # CORS middleware setup
| uploads/ # Video file storage

```

#### Technology Stack Details

##### Frontend Technologies

###### 1. React Native 0.74.5

- Cross-platform mobile development
- Hot reloading for development
- Native performance

###### 2. Expo SDK 51.0.28

- Managed workflow
- Built-in camera/media handling
- Easy deployment and testing

###### 3. React Navigation 6.x

- Stack navigation pattern
- Screen transition animations
- Route parameter passing

###### 4. Key Dependencies:

```

{
  "expo": "~51.0.28",
  "react": "18.2.0",
  "react-native": "0.74.5",
  "@react-navigation/native": "^6.1.7",
  "@react-navigation/stack": "^6.3.17",
  "expo-image-picker": "~15.0.7",
  "react-native-safe-area-context": "4.10.5"
}

```

##### Backend Technologies

###### 1. FastAPI

- Modern Python web framework
- Automatic API documentation
- Type hints and validation
- Async support

###### 2. SQLAlchemy

- ORM for database operations
- Migration support
- Relationship management

###### 3. Database Schema:

# Users table (optional)

- id (Integer, primary key)
- username (String, unique)

- email (String, unique)
- hashed\_password (String)
- created\_at (DateTime)
- # DrillResult table
- id (Integer, primary key)
- user\_id (String)
- analysis\_id (String, unique)
- drill\_type (String)
- count\_detected (Integer)
- duration (Float)
- confidence (Float)
- touches\_per\_minute (Float)
- average\_interval (Float)
- per\_foot\_counts (Text/JSON)
- detection\_details (Text/JSON)
- created\_at (DateTime)

#### 4. API Endpoints:

- GET /drill/available # List available drills
- POST /drill/analyze # Upload video for analysis
- GET /drill/status/{id} # Check analysis status
- GET /drill/results/{id} # Get analysis results
- GET /api/drills/{type}/progress # Get historical progress
- POST /auth/register # User registration (optional)
- POST /auth/login # User login (optional)
- GET /auth/me # Current user info (optional)

#### Data Flow

Recording & Analysis Flow

1. **User selects drill** → DrillSelectionScreen
2. **Countdown timer** → 3-2-1 countdown before camera opens
3. **Video recording** → 30-second fixed duration via expo-image-picker
4. **Upload video** → FormData POST to /drill/analyze
5. **Receive analysis\_id** → Unique identifier for tracking
6. **Poll status** → Check /drill/status/{id} every 2 seconds
7. **Retrieve results** → GET /drill/results/{id} when complete
8. **Save to database** → Automatic save via simple\_save\_integration
9. **Display results** → AnalysisResultsScreen with tabs

Progress Tracking Flow

1. **Navigate to progress** → From drill card "View Progress"
2. **Fetch historical data** → GET /api/drills/{type}/progress
3. **Display metrics:**
  - Last 7 days performance
  - Personal best
  - Recent sessions
  - Performance trends

#### UI/UX Design System

Color Palette

```
const Colors = {
  primary: '#004996', // Blue - Primary actions, headers
  gold: '#FCBF00',    // Gold - Highlights, badges
  white: '#FFFFFF',    // White - Backgrounds
  accent: '#E62644',  // Red - CTAs, important actions
  lightGray: '#F8F9FA', // Light Gray - Backgrounds
  darkGray: '#6C757D', // Dark Gray - Text, secondary
  navy: '#004996',    // Navy - Same as primary
};
```

Screen Designs

1. **HomeScreen:** Hero section with two main CTAs

2. **DrillSelectionScreen**: Card-based drill browser
3. **VideoRecordingScreen**: Countdown + recording tips
4. **AnalysisProgressScreen**: Live progress with status updates
5. **AnalysisResultsScreen**: Tabbed view (Overview/Detailed)
6. **DrillProgressScreen**: Charts and historical data

#### Key UI Components

- Consistent navigation headers
- Card-based layouts with shadows
- Progress bars and loading states
- Tab navigation for complex data
- Icon usage for visual clarity

#### **Security & Authentication**

##### Current Implementation

- **Optional Authentication**: JWT-based system
- **Anonymous Usage**: Default "anonymous" user\_id
- **Token Storage**: In-memory (not persistent)
- **CORS Configuration**: Allows mobile app origin
- **File Uploads**: Validated for video formats

##### Security Considerations

- Video files stored on server filesystem
- No HTTPS in development (use for production)
- SQL injection protection via SQLAlchemy
- Input validation on all endpoints

#### **Performance Optimizations**

##### Frontend

- **Lazy loading**: Screens loaded on navigation
- **Image optimization**: Using Expo's optimized image handling
- **Polling efficiency**: 2-second intervals with max attempts
- **State management**: Minimal re-renders

##### Backend

- **Async processing**: Video analysis in background threads
- **Database indexing**: On analysis\_id and user\_id
- **File cleanup**: Consider implementing video deletion
- **Response caching**: 15-minute cache on some endpoints

#### **Deployment & Infrastructure**

##### Current Setup

- **Backend Server**: DigitalOcean Droplet
  - IP: 147.182.224.87
  - Port: 8000
  - OS: Ubuntu
  - Access: SSH as root
- **Frontend Development**: Local Expo
  - Development: `npx expo start`
  - Clear cache: `npx expo start -c`
  - iOS/Android: Expo Go app

##### Environment Variables

- **Backend**: Hardcoded for development
- **Frontend**: API\_BASE\_URL in config.js

##### Server Management Commands

###### # SSH access

```
ssh root@147.182.224.87
```

###### # Virtual environment

```
source soccerapp-venv/bin/activate
```

###### # Start server

```
cd /root/soccerapp/backend
```

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

###### # Kill server

```
pkill -f "uvicorn:*8000"
```

## Testing Approach

Manual Testing Flow

1. Launch app: `npm expo start`
2. Test drill selection and recording
3. Verify 30-second enforcement
4. Check analysis completion
5. Validate results display
6. Test progress tracking

Known Issues & Solutions

- **CV Accuracy:** Currently detecting more touches than actual
- **Color undefined:** Fixed by adding Colors.navy
- **Timing inconsistency:** Standardized to 30 seconds
- **Build cache:** Clear with `npm expo start -c`

## Future Enhancements

Planned Features

1. **More Drills:** Passing, dribbling, shooting
2. **Video Playback:** Show analyzed video with overlays
3. **Social Features:** Share results, leaderboards
4. **Detailed Analytics:** Weekly/monthly reports
5. **Coach Mode:** Multi-player analysis

Technical Improvements

1. **Persistent Auth:** Secure token storage
2. **Push Notifications:** Analysis complete alerts
3. **Offline Support:** Queue uploads when offline
4. **Video Compression:** Reduce upload size
5. **Real-time Updates:** WebSocket for progress

Infrastructure Scaling

1. **HTTPS:** SSL certificate setup
2. **CDN:** For video delivery
3. **Cloud Storage:** S3 for videos
4. **Load Balancing:** Multiple backend instances
5. **Database:** PostgreSQL for production

## Development Commands

Frontend

```
# Start development
```

```
npm expo start
```

```
# Clear cache start
```

```
npm expo start -c
```

```
# Build for production
```

```
eas build --platform ios
```

```
eas build --platform android
```

Backend

```
# SSH to server
```

```
ssh root@147.182.224.87
```

```
# Activate virtual env
```

```
source soccerapp-venv/bin/activate
```

```
# Start server
```

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

```
# View logs
```

```
tail -f uvicorn.log
```

```
# Database operations
```

```
python
```

```
>>> from database import SessionLocal, engine
```

```
>>> from models import DrillResult
```

```
>>> db = SessionLocal()
```

```
>>> results = db.query(DrillResult).all()
```

## Business Logic

### Drill Analysis Scoring

- **Overall Score:** 0-100 composite score
- **Technique Score:** Form and execution
- **Consistency Score:** Rhythm and timing
- **Speed Score:** Touches per minute
- **Accuracy Score:** Detection confidence

### Progress Metrics

- **Personal Best:** Highest score achieved
- **Trend Analysis:** Last 7 days performance
- **Session History:** Individual attempt details
- **Improvement Tracking:** Score progression

This comprehensive documentation provides full context for the Soccer Training App, covering all technical aspects, architecture decisions, and implementation details for effective brainstorming and strategic planning.